TP #3 – GESTION D'ORDONNANCEMENT

8INF259 – Structure de données

Thématique

Dans ce projet, vous allez mettre en pratique vos compétences en structures de données dans un contexte industriel simulé. Vous devez modéliser et gérer un système d'ordonnancement de dispositifs de sécurité pour le verrouillage de machines (*lockout/tagout*), une procédure critique en milieu industriel.

Chaque machine comporte plusieurs dispositifs d'isolation à verrouiller, et certains dispositifs peuvent être partagés par plusieurs machines. La difficulté réside dans :

- Le respect des dépendances entre machines : certaines doivent être verrouillées avant d'autres (graphe orienté acyclique)
- La détection d'erreurs dans les données (cycles de dépendances)
- L'ordonnancement intelligent des opérations pour minimiser le temps total, en tenant compte des capacités limitées des équipes
- La gestion de la redondance : un dispositif partagé ne doit être verrouillé qu'une seule fois

Ce projet vous confronte aux problématiques de la lecture et du traitement de données réelles au format JSON, de la gestion efficace de la mémoire, de la modélisation par graphes et de la mise en œuvre d'algorithmes d'ordonnancement dans un environnement aux données variables.

Objectifs pédagogiques

Ce TP vous permettra d'atteindre plusieurs objectifs pédagogiques clés :

1. **Modélisation de données réelles**

Lire et interpréter des fichiers JSON pour créer en RAM les représentations des machines, des dispositifs et des équipes. Utiliser des structures de données appropriées (comme les `UNORDERED_MAP`) pour garantir un accès rapide aux éléments.

2. **Modélisation par graphe et parcours**

Construire un graphe orienté à partir des dépendances entre machines. Implémenter un parcours en largeur (BFS) pour déterminer un ordre de traitement valide (tri topologique). Détecter les erreurs de configuration (cycles) dans le graphe.

3. **Ordonnancement sous contraintes**

Concevoir un algorithme glouton pour répartir les opérations entre les équipes en tenant compte de leurs capacités (types de devices gérés, nombre maximum par machine) et en minimisant le temps total d'exécution (makespan).

4. **Gestion de la redondance et robustesse**

Veiller à ce qu'un même dispositif ne soit verrouillé qu'une seule fois, même lorsqu'il est commun à plusieurs machines. Assurer la cohérence du système face aux variations dans les données d'entrée (nombre variable de machines, de dispositifs, d'équipes, présence ou absence de certains champs JSON).3. Ordonnancement sous contraintes

Données d'entrée

Le programme lit trois fichiers JSON situés dans le dossier `./assets/` :

- **devices.json** — Les dispositifs de sécurité

Chaque device possède un `id` (entier unique), un `type` (chaîne, ex: "VALVE", "BREAKER"), un `name` (chaîne descriptive) et un `lock_time` (entier, temps de verrouillage en unités abstraites). Le champ `lock_time` peut être absent ; dans ce cas, la valeur par défaut est 1.

- **machines.json** — Les machines industrielles

Chaque machine possède un `id`, un `name`, un tableau `related_devices` (IDs des devices à verrouiller) et un tableau `depends_on` (IDs des machines prérequises). Le champ `depends_on` peut être absent ; dans ce cas, la machine n'a aucun prérequis.

- **teams.json** — Les équipes de techniciens

Chaque équipe possède un `name`, un tableau `capabilities` (les types de devices qu'elle peut verrouiller) et un `max_capacity` (nombre max de devices par machine). Le champ `max_capacity` peut être absent ; dans ce cas, la capacité est illimitée.

Fichiers fournis

Vous recevez les fichiers suivants :

- `Device.h` — Classe Device (complète, avec getters et signature de la méthode statique)
- `Machine.h` — Classe Machine (complète)
- `Team.h` — Classe Team (complète, inclut la méthode `canHandle`)

- SCHEDULER.H — Classe Scheduler (**section privée vide** : à vous de la compléter)
- MAIN.CPP — Point d'entrée (**ne doit pas être modifié**)
- INCLUDE/JSON.HPP — Bibliothèque nlohmann/json
- ASSETS/ — Dossier contenant les fichiers JSON

Vous devez créer les fichiers .CPP correspondants : DEVICE.CPP, MACHINE.CPP, TEAM.CPP et SCHEDULER.CPP.

Vous êtes libres d'ajouter des structs, des classes auxiliaires ou des méthodes privées dans SCHEDULER.H, mais vous ne devez **ni modifier les signatures publiques existantes, ni modifier le MAIN.CPP**.

Attentes fonctionnelles

1. **Classe Device (implémenter DEVICE.CPP)**

Lire le fichier DEVICES.JSON et créer en RAM autant de devices que présent dans le fichier. Retourner un UNORDERED_MAP<INT, DEVICE*> pour un accès O(1) par ID.

2. **Classe Machine (implémenter MACHINE.CPP)**

Lire le fichier MACHINES.JSON et créer en RAM autant de machines que présent dans le fichier. Convertir les IDs du champ RELATED_DEVICES en pointeurs DEVICE* à l'aide de la map de devices reçue en paramètre. Lire le champ DEPENDS_ON s'il est présent.

3. **Classe Team (implémenter TEAM.CPP)**

Lire le fichier TEAMS.JSON et créer en RAM autant d'équipes que présent dans le fichier. Implémenter la méthode CANHANDLE(DEVICETYPE).

4. **Classe Scheduler (implémenter SCHEDULER.CPP et compléter SCHEDULER.H)**

C'est la classe qui demande le plus de travail. Vous devez implémenter toutes les méthodes publiques décrites ci-dessous **et définir vous-mêmes les attributs privés et méthodes utilitaires nécessaires**.

displayNotConnectedDevices()

Affiche les devices qui ne sont référencés par aucune machine.

displayTeams()

Affiche les équipes avec leurs capabilities et leur capacité maximale.

buildDependencyGraph()

Construit un graphe orienté à partir des dépendances entre machines. Si machine B a DEPENDS_ON: [A], cela signifie que A doit être traitée avant B (arête A → B dans le graphe).

hasCycle()

Détecte la présence d'un cycle dans le graphe de dépendances. S'il y a un cycle, le programme doit afficher un message d'erreur et quitter avec le code 1 (voir le MAIN.CPP).

topologicalSort(tieBreaker)

Détermine l'ordre de verrouillage des machines en respectant les dépendances. L'approche recommandée est un **parcours en largeur (BFS)** : on part des machines sans prérequis, on les traite, puis on libère leurs dépendants. Le paramètre TIEBREAKER est une lambda qui détermine l'ordre entre deux machines prêtes simultanément.

schedule()

Pour chaque machine (dans l'ordre topologique), pour chaque device de la machine :

- Si le device a **déjà été verrouillé** par une machine précédente → on le passe.
- Sinon, on l'assigne à la **meilleure équipe disponible** :
  - L'équipe doit être **capable** de gérer ce type de device
  - L'équipe ne doit pas avoir **dépassé sa capacité** pour cette machine
  - Parmi les candidates, on choisit celle avec la **charge totale la plus faible** (pour équilibrer le travail et minimiser le makespan)

Le temps de début d'un device = temps total actuel de l'équipe. Le temps de fin = début + LOCKTIME.

getMakespan()

Retourne le makespan : le maximum des temps totaux de toutes les équipes.

displaySummary()

Affiche l'ordre topologique des machines, puis un résumé par machine (nombre de devices verrouillés, nombre ignorés, lesquels ont été ignorés).

displayGantt()

Affiche un diagramme de Gantt en colonnes :

- Chaque colonne = une équipe
- Chaque ligne = une unité de temps (T=0, T=1, ...)
- Chaque cellule = le nom du device en cours, suivi de son ID entre parenthèses
- Le guillemet " indique la continuation du même device (lockTime > 1)
- Une cellule vide = l'équipe est inactive

displayStats()

Affiche les statistiques finales : makespan, charge et nombre de devices par équipe, équipe goulot d'étranglement (bottleneck), nombre total de devices verrouillés, nombre de skips, et efficacité (%).

Exemple de sortie

Pour les fichiers JSON de base fournis, voici un extrait de la sortie attendue :

```
=== DEVICES NOT CONNECTED TO ANY MACHINE ===
- Device 18 ( Safety Relay A ) is not connected.

=== TEAMS ===
- Alpha (capacity=4) : BREAKER, CIRCUIT_BREAKER, DISCONNECT_SWITCH, TAG, PADLOCK, FUSE
- Beta (capacity=3) : VALVE, PRESSURE_GAUGE, FLOW_METER, EMERGENCY_STOP
- Gamma (capacity=3) : CONTROL_SWITCH, SAFETY_RELAY, SAFETY_LOCK, PADLOCK, TAG

=== TOPOLOGICAL ORDER ===
  1 : Boiler A
  2 : Boiler B
  3 : Main Pump  [after: 1]
  4 : Secondary Pump  [after: 2]
  ...

=== LOCKOUT SUMMARY (by Machine) ===
Machine 1 : Boiler A  -- 6 locked, 0 skipped
Machine 3 : Main Pump  -- 5 locked, 1 skipped
   Skipped: 3(Main Circuit Breaker)
...

=== GANTT ===
  T | Alpha                          | Beta                      | Gamma
----+--------------------------------+---------------------------+----------------------------
  0 | Main Circuit Breaker(3)        | Inlet Valve A(1)          | Boiler Lockout Tag(30)
  1 |   "                            |   "                       | Boiler B Padlock(44)
  2 |   "                            |   "                       |   "
  3 |   "                            |   "                       | Secondary Padlock(19)
  ...

=== STATISTICS ===
MAKESPAN : 90 time units

  Team Alpha : 27 devices, total_time=90, max_capacity=4  *** BOTTLENECK ***
  Team Beta : 17 devices, total_time=53, max_capacity=3
  Team Gamma : 26 devices, total_time=62, max_capacity=3

  Devices locked  : 70 / 75
  Already-locked skips : 20
  Efficiency : 75.9% (useful_time / total_available_time)
```

La sortie complète attendue pour chaque jeu de données sera fournie séparément (dans le fichier d'assets sur Moodle).

Robustesse

Votre programme doit fonctionner correctement avec des fichiers JSON différents de ceux fournis. Je modifierai uniquement le contenu des fichiers dans ASSETS/ :

- Plus ou moins de machines, de devices, d'équipes
- Des capacités différentes (y compris absentes = illimitées)
- Des LOCK_TIME différents (y compris absents = valeur 1)
- Des dépendances différentes (y compris absentes = pas de dépendances)
- Des cycles dans les dépendances (le programme doit les détecter et quitter proprement)
- Des devices référencés par aucune machine (orphelins)
- Des devices partagés entre plus ou moins de machines

Format de rendu

Pour être valide, il faut rendre un unique fichier .zip nommé **TP3_NOM_PRENOM.zip**.

Il est impératif que l'ensemble des fichiers soient au même endroit dans le zip que dans le fichier fourni. Les fichiers doivent être directement dans le dossier zip, au même format que le fichier .zip envoyé.

ATTENTION : Une partie de la note est 100% automatisée, donc il faut impérativement que le format soit tel que demandé.

Date de rendu

Le projet devra être remis au plus tard le **17 avril 2026 à 23h59** sur Moodle via l'option créée à cet effet.

Conseils

- **Décomposez le problème** : implémentez d'abord Device, Machine et Team (parsing JSON). Testez. Puis attaquez le Scheduler méthode par méthode.
- **Testez avec le petit jeu de données d'abord** : il est plus facile à vérifier manuellement.
- **Pensez aux structures de données** : le choix entre UNORDERED_MAP, UNORDERED_SET, VECTOR, QUEUE, PRIORITY_QUEUE n'est pas arbitraire — chaque structure a des forces et des faiblesses. Choisissez en fonction de ce que vous devez faire (accès par clé ? parcours séquentiel ? insertion ordonnée ?).
- **Le Gantt est un bon outil de débogage** : si votre ordonnancement est incorrect, le Gantt le rendra visible immédiatement.
- **Gérez les cas limites** : champs absents dans le JSON, devices sans machine, cycles...

Grille de notation

Composante	Notation	Détails
device.cpp	10 %	Lecture JSON, construction de la map, gestion du lock_time absent
machine.cpp	10 %	Lecture JSON, résolution IDs → pointeurs, gestion du depends_on absent
team.cpp	5 %	Lecture JSON, canHandle, gestion du max_capacity absent
Scheduler – displayTeams + displayNotConnectedDevices	5 %	Identification correcte des orphelins, affichage des équipes
Scheduler – hasCycle + buildDependencyGraph	15 %	Graphe correct, détection de cycle fonctionnelle
Scheduler – topologicalSort	15 %	Ordre valide respectant les dépendances, tie-breaking correct
Scheduler – schedule	20 %	Assignation correcte (capabilities, capacité, charge), devices partagés, makespan correct
Scheduler – displaySummary + displayGantt + displayStats	5 %	Affichage conforme à la sortie attendue
Robustesse	15 %	Comportement correct avec des JSON modifiés (cf. section Robustesse)
Total	**100 %**	